# Práctica 1: Emmagatzematge de Dades en un Sistema d'Informació Cinematogràfic

Ivan Morillas, Yani Aici

Fecha de entrega: 17 de marzo de 2024

# Contents

**1 Introducción** **3**

**2 Fase 1: Diseño de una base de datos relacional** **4**
2.1 Descripción del Diseño . . . . . 4
2.2 Gestor de Base de Datos Utilizado . . . . . 4
2.3 Definición de Tablas y Campos . . . . . 5
2.4 Definición de Claves y Índices . . . . . 8
2.5 Normalización . . . . . 9

**3 Fase 2: Importación de Datos** **10**
3.1 Proceso de Importación . . . . . 10
3.2 Tecnología Utilizada . . . . . 18
3.3 Desafíos y Soluciones . . . . . 18
3.4 Resultados de la importación . . . . . 19

**4 Fase 3: Construcción de una Red Cinematográfica** **24**
4.1 Diseño de la Base de Datos Orientada a Grafos . . . . . 24
4.2 Importación de Datos . . . . . 27

# 1 Introducción

Esta documentación detalla el desarrollo de una base de datos relacional diseñada para el almacenamiento y gestión de datos cinematográficos para FilmLib. Se aborda el proceso desde el diseño inicial, basado en un modelo entidad-relación y un modelo relacional, hasta la implementación utilizando un gestor de base de datos específico.

# 2 Fase 1: Diseño de una base de datos relacional

## 2.1 Descripción del Diseño

En el modelo Entidad-Relación, las entidades están formadas principalmente por **Persons** y **Titles**, pero como **Titles** tiene de atributos una lista de **Genres** y de **Countries** y en **Persons** tiene el atributo de **Roles**, el cual puede tener 2 valores diferentes pero no a la vez, pues se ha creado una Entidad de cada uno de ellos y también hemos creado una Entidad de los proveedores de las plataformas Streamings y para acabar hemos creado una Entidad de peliculas duplicadas, que explicaremos mas adelante, entonces tendriamos un total de 7 Entidades que son: **Person**, **Titles**, **Genres**, **Countries**, **Roles**, **Streamings** y **TitleDuplicated**.
De relaciones, primeramente, teniamos que 1 **Title** tiene muchas **Persons** y 1 **Person** puede estar en muchas **Titles**. Pero, como anteriormente habiamos creado 7 Entidades, hemos relacionado la mayoria de Entidades donde hemos obtenido un total de 4 Relaciones que son: **TitlesPersonsRoles**, **TitleCountries**, **TitleGenres**, **TitleStreamings**.

## 2.2 Gestor de Base de Datos Utilizado

Una de las principales razones por la que hemos elegido MySQL como gestor de base de datos era porque años anteriores habiamos tocado este gestor en la asignatura **Base de Dades** y era la que más conociamos y porque es fácil de programar y también es uno de los mejores gestores.

Figure 1: Logo MySQL

## 2.3 Definición de Tablas y Campos

Como anteriormente hemos explicado, habiamos creado 7 Entidades, por lo que hemos creado 7 tablas con sus respectivos atributos.

La tabla **Titles** tiene de atributos: **titleId** (String), **title** (String), **titleType** (String), **titleDescription** (String), **titleReleaseYear** (Int), **titleAgeCertification** (String), **titleRuntime** (Int), **titleSeasons** (Float), **titleImdbId** (String), **titleImdbScore** (Float), **titleImdbVotes** (Float), **titleTmdbPopularity0** (Float) y **titleTmdbScore** (Float).

```sql
CREATE TABLE titles(
    title_id varchar(255) not null unique primary key,
    title varchar(255) not null,
    title_type varchar(255) not null,
    title_description text,
    title_release_year int,
    title_age_certification varchar(255),
    title_runtime int,
    title_seasons float,
    title_imdb_id varchar(255),
    title_imdb_score float,
    title_imdb_votes float,
    title_tmdb_popularity float,
    title_tmdb_score float
);
```

Figure 2: Table Titles

La tabla **Persons** tiene de atributos: **personId** (Int), **titleId** (String), **personName** (String), **personCharacter** (String), **personRole** (String).

```
CREATE TABLE persons(
    person_id int not null unique primary key,
    title_id varchar(255) not null,
    person_name varchar(255) not null,
    person_character text,
    person_role varchar(255) not null
);
```

Figure 3: Table Persons

La tabla **Genres** tiene de atributos: **genreId** (Int) y **genreName** (String).

```
CREATE TABLE genres (
    genre_id int auto_increment primary key,
    genre_name varchar(255) not null unique
);
```

Figure 4: Table Genres

La tabla **Countries** tiene de atributos: **contryId** (Int) y **contryName** (String).

```
CREATE TABLE countries (
    country_id int auto_increment primary key,
    country_name varchar(255) not null unique
);
```

Figure 5: Table Countries

La tabla **Roles** tiene de atributos: **roleId** (Int) y **roleName** (String).

```sql
CREATE TABLE roles (
    role_id int auto_increment primary key,
    role_name varchar(255) not null unique
);
```

Figure 6: Table Roles

La tabla **Streamings** tiene de atributos: **streamingId** (Int) y **streamingName** (String).

```sql
CREATE TABLE streamings (
    streaming_id int auto_increment primary key,
    streaming_name varchar(255) not null unique
);
```

Figure 7: Table Streamings

La tabla **TitlesDuplicated** tiene los mismos atributos que **Titles** ya que es una tabla donde guardamos peliculas que se repiten en diferentes plataformas de **Streaming**.

```sql
CREATE TABLE titles_duplicated(
    title_id varchar(255) not null unique primary key,
    title varchar(255) not null,
    title_type varchar(255) not null,
    title_description text,
    title_release_year int,
    title_age_certification varchar(255),
    title_runtime int,
    title_seasons float,
    title_imdb_id varchar(255),
    title_imdb_score float,
    title_imdb_votes float,
    title_tmdb_popularity float,
    title_tmdb_score float
);
```

Figure 8: Table TitlesDuplicated

## 2.4 Definición de Claves y Índices

A la hora de hacer las relaciones, primero hemos creado sus respectivas tablas para almacenar la relación que hay entre entidades y, para que esa relacion sea efectiva, declaramos para cada relación una clave primaria (**Primary Key**) para cada entidad y sus respectivas claves foráneas (**Foreign Key**) para hacer la referencia de dicha relación.

A la hora de añadir las respectivas IDs para cada elemento de la tabla, en las tablas **Titles**, **Person** y **TitlesDuplicated** no habia problema, porque se importaban, pero en las otras tablas, hemos usado un incrementador el cual se le añadia automaticamente una ID cada vez que se añadia un elemento en la tabla.

La relación **TitlesPersonsRoles** tiene de clave primaria **personId**, **titleId** y **roleId** y de clave foránea **personId** ⇒ **Persons(personId)**, **titleId** ⇒ **Titles(titleId)** y **roleId** ⇒ **Roles(roleId)**.

```sql
CREATE TABLE title_person (
    person_id int not null,
    title_id varchar(255) not null,
    role_id int not null,
    primary key (person_id, title_id, role_id),
    foreign key (person_id) references persons(person_id),
    foreign key (title_id) references titles(title_id),
    foreign key (role_id) references roles(role_id)
);
```

Figure 9: Table TitlesPersonsRoles

La relación **TitlesGenres** tiene de clave primaria **titleId** y **genreId** y de clave foránea **titleId** ⇒ **Titles(titleId)** y **genreId** ⇒ **Genres(genreId)**.

```sql
CREATE TABLE titles_genres (
    title_id varchar(255) not null,
    genre_id int not null,
    primary key (title_id, genre_id),
    foreign key (title_id) references titles(title_id),
    foreign key (genre_id) references genres(genre_id)
);
```

Figure 10: Table TitlesGenres

La relación **TitlesCountries** tiene de clave primaria **titleId** y **countryId** y de clave foránea **titleId** ⇒ **Titles(titleId)** y **countryId** ⇒ **Countries(countryId)**.

```sql
CREATE TABLE titles_countries (
    title_id varchar(255) not null,
    country_id int not null,
    primary key (title_id, country_id),
    foreign key (title_id) references titles(title_id),
    foreign key (country_id) references countries(country_id)
);
```

Figure 11: Table TitlesCountries

La relación **TitlesStreamings** tiene de clave primaria **titleId** y **streamingId** y de clave foránea **titleId** ⇒ **Titles(titleId)** y **streamingId** ⇒ **Streamings(streamingId)**.

```sql
CREATE TABLE titles_streamings (
    title_id varchar(255) not null,
    streaming_id int not null,
    primary key (title_id, streaming_id),
    foreign key (title_id) references titles(title_id),
    foreign key (streaming_id) references streamings(streaming_id)
);
```

Figure 12: Table TitlesStreamings

## 2.5 Normalización

Como habian **Titles** que se repetian de diferentes **Streamings**, habian algunos que tenian la misma información, entonces estos no los hemos guardado porque seria innecesario tener una base de datos con varios datos que son iguales, no es eficaz para este tipo de base de datos, por lo que no los guardamos, en cambio, hay **Titles** que tienen la misma ID y el mismo nombre pero con otros datos diferentes, por lo que no sabiamos cual coger entre todos estos. Se nos ocurrió crear una tabla extra, sin que tenga relación con las otras creadas anteriormente, para almacenar los **TitlesDuplicated** y que en un posible futuro se puedan gestionar como el usuario quiera.

# 3 Fase 2: Importación de Datos

## 3.1 Proceso de Importación

Primero de todo, hemos conectado el proyecto de **JavaSpring** con nuestra base de datos **MySQL** para poder importar los datos, configurando el puerto, el cual hemos usado el default (3306) y el nombre de nuestra base de datos (dataset).

```
spring.datasource.url=jdbc:mysql://localhost:3306/dataset
spring.datasource.username=root
spring.datasource.password=password1234
spring.jpa.hibernate.ddl-auto=update
```

Figure 13: Configuración para la conexión con la BD

A la hora de importar los .csv de todas las plataformas de streaming, hemos creado una clase para cada tabla creada con **MySQL** con sus respectivos atributos y así poder guardar mejor los datos importados.

```
2 usages   Ivan Morillas *
@Entity
public class Persons {
    4 usages
    @Id
    private int personId;
    4 usages
    private String titleId;
    4 usages
    private String personName;
    4 usages
    private String personCharacter;
    4 usages
    private String personRole;
    1 usage   Ivan Morillas
    public Persons(int personId, String titleId, String name, String character, String role){
        this.personId = personId;
        this.titleId = titleId;
        this.personName = name;
        this.personCharacter = character;
        this.personRole = role;
    }
```

Figure 14: Clase Person de Java

En esta clase **Persons**, hemos declarado sus atributos con sus respectivos tipos y su constructor. Podemos ver un **@Entity** que sirve para identificar el nombre de la tabla en nuestra BD de **MySQL** y un **@Id** para marcar cual es su ID.
Esta estructura es la misma con las clases **Titles** y **TitlesDuplicated** pero con sus atributos juntamente con los Getters y Setters.

```
3 usages   Ivan Morillas *
@Entity
public class Streamings {
    3 usages
    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    private int streamingId;
    4 usages
    private String streamingName;
    1 usage   Ivan Morillas
    public Streamings(String streamingName) { this.streamingName = streamingName; }
```

Figure 15: Clase Streaming de Java

Para la clase Streaming, tiene una estructura similar a la estructura explicada anteriormente, pero con una ligera diferencia y es la generación del ID. Antes, la ID se obtenia a partir de la importación de los .csv, pero, tanto la clase **Streamings**, como **Genres**, **Countries** y **Roles**, el ID no lo podiamos obtener a partir de la importación de los .csv, así que hemos usado **@GeneratedValue** para que cree una ID automatica cada vez que se añada un elemento en la tabla.

```java
@Entity
public class TitlePerson {
    3 usages
    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    int titlePersonId;
    4 usages
    int personId;
    4 usages
    String titleId;
    4 usages
    int roleId;
    1 usage   Ivan Morillas *
    public TitlePerson(int personId, String titleId, int roleId){
        this.personId = personId;
        this.titleId = titleId;
        this.roleId = roleId;
    }
```

Figure 16: Clase TitlePersonRole de Java

Para las clases de las relaciones, tendrian la misma estructura que las anteriores pero teniendo de atributos las IDs de las clases que vamos a relacionar y su constructor.
Después de crear las clases, seguidamente debiamos crear una interfaz de cada una de las clases con **CrudRepository** para poder realizar operaciones CRUD (Crear, Leer, Actualizar, Eliminar).

```java
public interface ITitle extends CrudRepository<Titles, String> {

}
```

Figure 17: Interfaz Title de Java

En la Figure 17, podemos ver un ejemplo de como son las interfaces de las clases entidades con el **CrudRepository** especificando la clase a la cual hace referencia y el tipo de su ID.

```java
public interface ITitlesGenres extends CrudRepository<TitlesGenres, Integer> {
    1 usage  Ivan Morillas
    boolean existsByGenreIdAndTitleId(Integer genreId, String titleId);
}
```

Figure 18: Interfaz TitleGenre de Java

Y en la figure 18, se ve un ejemplo de las interfaces de las clases relacion donde tiene el **CrudRepository** con su respectiva clase y el tipo de su ID y con unas funciones que más adelante vamos a explicar.

Para empezar a importar todos los .csv, los hemos guardado en una carpeta, en la cual leemos todos los nombres de todos los .csv y almacenar en una lista todos los nombres de los proveedores.

```java
FileReader fileCSV = null;
CSVReader csvReader = null;
List<String> titleIds = new ArrayList<>();
List<String> titleNames = new ArrayList<>();
List<Integer> streamingIds = new ArrayList<>();
String nameFile;
File directory = new File( pathname: "C:\\Users\\ivanm\\OneDrive\\Escritorio\\Universidad\\4°\\SI0\\Practica\\Practica1\\Dataset\\files");
if(!directory.exists()){
    System.out.println("Directory not found!");
}
File[] files = directory.listFiles();
for(File file : files) {
    String completeName = file.getName();
    int index = completeName.lastIndexOf( str: "_");
    nameFile = completeName.substring(0, index);
    if (repositoryStreamings.findByStreamingName(nameFile) == null) {
        titleNames.add(nameFile);
        repositoryStreamings.save(new Streamings(nameFile));
        streamingIds.add(repositoryStreamings.findByStreamingName(nameFile).getStreamingId());
    }
}
```

Figure 19: Funcion para almacenar los nombres de los proveedores

En la Figure 19, especificamos la ruta de la ubicación de la carpeta donde tenemos todos los .csv almacenados, extraemos solo el nombre de los ficheros y los almacenamos en la tabla **Streamings** para poder trabajar con ellos más adelante.

```
for(String name : titleNames) {
    try {
        fileCSV = new FileReader( fileName: "C:\\Users\\ivanm\\OneDrive\\Escritorio\\Universidad\\4º\\SIO\\Practica\\Practica1\\Dataset\\files\\" + name + "_Titles.csv");
```

Figure 20: Bucle para leer todos los ficheros .csv

Para empezar a leer todos los ficheros, vamos leyendo todos los nombres almacenados anteriormente y especificamos el fichero que queremos leer.

```
CSVParser puntoYcoma = new CSVParserBuilder().withSeparator(',').build();
csvReader = new CSVReaderBuilder(fileCSV).withCSVParser(puntoYcoma).build();
csvReader.readNext();
while ((line = csvReader.readNext()) != null) {
    String genresString = line[7];
    List<String> genres;
    List<Integer> genreIds = new ArrayList<>();
    if (genresString.isEmpty()) {
        genres = null;
    } else if (genresString.length() < 4) {
        genres = new ArrayList<>();
    } else {
        String[] genreArray = genresString.substring(1, genresString.length() - 1).split( regex: "(?<!\\\\),");
        genres = Arrays.stream(genreArray)
                .map(genre -> genre.replace( target: "'",  replacement: ""))
                .map(genre -> genre.replace( target: " ",  replacement: ""))
                .collect(Collectors.toList());
    }
    if (genres != null) {
        for (String genre : genres) {
            Genres genreObject = (Genres) repositoryGenres.findByGenreName(genre);
            if (genreObject == null) {
                genreObject = new Genres(genre);
                repositoryGenres.save(genreObject);
            }
            genreIds.add(genreObject.getGenreId());
        }
    }
```

Figure 21: Estructura para almacenar los Genres

Para empezar a almacenar todos los **Titles**, miramos primero en almacenar las listas de Genre y Country que eran un problema principal para guardar los Titles.
Con **CSVParser**, declaramos que el separador es **','** y empezamos a leer a partir de la 2a linea, ya que la primera línea son los nombres de los campos. Con el **While** empezamos a leer todo el .csv y el campo [**7**] es donde está la lista de los **Genres**, donde primero comprobamos si está vacío o no, luego si tiene una lista vacia y por ultimo si tiene contenido. Si tiene contenido, extraemos solo los nombres de los **Genres** y los almacenamos en la tabla de **Genres**.

```java
String countriesString = line[8];
List<String> countries;
List<Integer> countryIds = new ArrayList<>();
if (countriesString.isEmpty()) {
    countries = null;
} else if (countriesString.length() < 4) {
    countries = new ArrayList<>();
} else {
    String[] countryArray = countriesString.substring(1, countriesString.length() - 1).split( regex: "(?<!\\\\),");
    countries = Arrays.stream(countryArray)
            .map(country -> country.replace( target: "'",  replacement: ""))
            .map(country -> country.replace( target: " ",  replacement: ""))
            .collect(Collectors.toList());
}
if (countries != null) {
    for (String country : countries) {
        Countries CountryObject = (Countries) repositoryCountries.findByCountryName(country);
        if (CountryObject == null) {
            CountryObject = new Countries(country);
            repositoryCountries.save(CountryObject);
        }
        countryIds.add(CountryObject.getCountryId());
    }
}
```

Figure 22: Estructura para almacenar los Countries

En la Figure 22 se puede ver que tiene la misma estructura que la Figure 21 pero en vez de trabajar con el campo [**7**], es con el [**8**] que es el que contiene la lista de los **Countries**.

```java
String seasons = line[9];
String imdb_score = line[11];
String imdb_votes = line[12];
String tmdb_popularity = line[13];
String tmdb_score = line[14];
if (seasons.isEmpty()) {
    seasons = "0.0";
}
if (imdb_score.isEmpty()) {
    imdb_score = "0.0";
}
if (imdb_votes.isEmpty()) {
    imdb_votes = "0.0";
}
if (tmdb_popularity.isEmpty()) {
    tmdb_popularity = "0.0";
}
if (tmdb_score.isEmpty()) {
    tmdb_score = "0.0";
}
```

Figure 23: Gestión de campos vacíos

Para los campos que son de tipo **Float** y pueden estar vacíos a la hora de leer los .csv, los hemos inicializado con **0.0**.

```java
if (!repositoryTitle.existsById(line[0])) {
    repositoryTitle.save(new Titles(line[0], line[1], line[2], line[3], Integer.parseInt(line[4]),
            line[5], Integer.parseInt(line[6]), Double.parseDouble(seasons),
            line[10], Double.parseDouble(imdb_score), Double.parseDouble(imdb_votes),
            Double.parseDouble(tmdb_popularity), Double.parseDouble(tmdb_score)));
}else{
    Titles originalTitle = repositoryTitle.findById(line[0]).get();
    boolean areEquals = compareData(line, originalTitle);
    if(!areEquals){
        repositoryTitlesDuplicated.save(new TitlesDuplicated(line[0], line[1], line[2], line[3], Integer.parseInt(line[4]),
                line[5], Integer.parseInt(line[6]), Double.parseDouble(seasons),
                line[10], Double.parseDouble(imdb_score), Double.parseDouble(imdb_votes),
                Double.parseDouble(tmdb_popularity), Double.parseDouble(tmdb_score)));
    }
}
```

Figure 24: Estructura para almacenar los Titles y TitlesDuplicated

Una vez tratado los campos, los cuales podrian llevar problemas si no se tratan adecuadamente, podemos almacenar las entidades **Titles** en nuestra tabla. Si vemos que el **Titles** se repite con la misma ID, lo comparamos con el **Titles** que tenemos almacenado y vemos que si es exactamente igual al que tenemos almacenado, no lo guardamos, pero si tiene el mismo ID pero con valores diferentes, lo almacenamos en nuestra tabla **TitlesDuplicated**.

```java
for (Integer genreId : genreIds) {
    if (!repositoryTitleGenre.existsByGenreIdAndTitleId(genreId, line[0])) {
        TitlesGenres titleGenre = new TitlesGenres(line[0], genreId);
        repositoryTitleGenre.save(titleGenre);
    }
}
for (Integer countryId : countryIds) {
    if (!repositoryTitleCountry.existsByCountryIdAndTitleId(countryId, line[0])) {
        TitlesCountries titleCountry = new TitlesCountries(line[0], countryId);
        repositoryTitleCountry.save(titleCountry);
    }
}
for (Integer streamingId : streamingIds) {
    if(streamingId == repositoryStreamings.findByStreamingName(name).getStreamingId()){
        if (!repositoryTitlesStreamings.existsByStreamingIdAndTitleId(streamingId, line[0])) {
            TitlesStreamings streamingTitle = new TitlesStreamings(line[0], streamingId);
            repositoryTitlesStreamings.save(streamingTitle);
        }
    }
}
```

Figure 25: Estructura para almacenar las relaciones TitleGenre, TitleCountry y TitleStreaming

Con estos bucles, almacenamos las relaciones que tiene los **Titles** con sus **Genres**, **Countries** y **Streamings**. Primero se mira si existe dicha relacion en nuestra BD, sino, se almacena.

```java
try {
    fileCSV = new FileReader( fileName: "C:\\Users\\ivanm\\OneDrive\\Escritorio\\Universidad\\4°\\SI0\\Practica\\Practica1\\Dataset\\files\\" + name + "_Credits.csv");
    Long total = Files.lines(Paths.get( first: "C:\\Users\\ivanm\\OneDrive\\Escritorio\\Universidad\\4°\\SI0\\Practica\\Practica1\\Dataset\\files\\" + name + "_Credits.
    System.out.println(name + "_Credits.csv reading...");
    int progress = 0;
    CSVParser puntoYcoma = new CSVParserBuilder().withSeparator(',').build();
    csvReader = new CSVReaderBuilder(fileCSV).withCSVParser(puntoYcoma).build();
    String[] line = null;
    csvReader.readNext();
    while ((line = csvReader.readNext()) != null) {
        repositoryPerson.save(new Persons(Integer.parseInt(line[0]), line[1], line[2], line[3], line[4]));
        String roleString = line[4];
        List<String> roles = new ArrayList<>();
        List<Integer> rolesIds = new ArrayList<>();
        if (roleString.isEmpty()) {
            roles = null;
        } else {
            roles.add(roleString);
        }
        if (roles != null) {
            for (String role : roles) {
                Roles roleObject = (Roles) repositoryRoles.findByRoleName(role);
                if (roleObject == null) {
                    roleObject = new Roles(role);
                    repositoryRoles.save(roleObject);
                }
                rolesIds.add(roleObject.getRoleId());
            }
        }
```

Figure 26: Estructura para almacenar Persons y Roles

La estructura para almacenar la entidad **Persons**, es similar a la de **Titles**, solo que sus campos no hacen falta tratarlos. Almacenamos los datos directamente en la tabla de **Persons** y después cogemos el campo [**4**], que es el que contiene el **Role**, y lo almacenamos en nuestra tabla de **Roles**.

```java
for (String title : titleIds) {
    if (line[1].equals(title)) {
        for (Integer roleId : rolesIds) {
            if (!repositoryTitlePerson.existsByPersonIdAndTitleIdAndRoleId(Integer.parseInt(line[0]), title, roleId)) {
                TitlePerson titlePerson = new TitlePerson(Integer.parseInt(line[0]), title, roleId);
                repositoryTitlePerson.save(titlePerson);
            }
        }
    }
}
```

Figure 27: Estructura para almacenar la relación TitlesPersonsRoles

En este último bucle, almacenamos la relación con **Title**, **Person** y **Role**, ya que nos dimos cuenta de que una **Persona** puede tiene 2 relaciones con el mismo **Title** como **Actor** y **Director**, por lo que tenian que ser relaciones diferentes, así que hicimos una relación de 3 entidades.

```java
private boolean compareData(String[] line, Titles originalTitle) {
    return (isEmptyOrEqual(line[1], originalTitle.getTitle()) &&
            isEmptyOrEqual(line[2], originalTitle.getTitleType()) &&
            isEmptyOrEqual(line[3], originalTitle.getTitleDescription()) &&
            (isEmpty(line[4]) || Integer.parseInt(line[4]) == originalTitle.getTitleReleaseYear()) &&
            isEmptyOrEqual(line[5], originalTitle.getTitleAgeCertification()) &&
            (isEmpty(line[6]) || Integer.parseInt(line[6]) == originalTitle.getTitleRuntime()) &&
            (isEmpty(line[9]) || Double.parseDouble(line[9]) == originalTitle.getTitleSeasons()) &&
            isEmptyOrEqual(line[10], originalTitle.getTitleImdbId()) &&
            (isEmpty(line[11]) || Double.parseDouble(line[11]) == originalTitle.getTitleImdbScore()) &&
            (isEmpty(line[12]) || Double.parseDouble(line[12]) == originalTitle.getTitleImdbVotes()) &&
            (isEmpty(line[13]) || Double.parseDouble(line[13]) == originalTitle.getTitleTmdbPopularity()) &&
            (isEmpty(line[14]) || Double.parseDouble(line[14]) == originalTitle.getTitleTmdbScore()));
}
5 usages  new *
private boolean isEmptyOrEqual(String value1, String value2) {
    return (value1 == null || value1.isEmpty()) ? value2 == null || value2.isEmpty() : value1.equals(value2);
}

7 usages  new *
private boolean isEmpty(String value) { return value == null || value.isEmpty(); }
```

Figure 28: Estructura para almacenar Persons y Roles

Y por ultimo, las funciones que hemos usado para comparar los **Titles** para saber si son exactamente iguales o no.

## 3.2 Tecnología Utilizada

Para la importación de los .csv que nos han proporcionado desde el Moodle, hemos pensado en utilizar **JavaSpring** que es un popular framework gratuito y de código abierto para poder construir aplicaciones en Java como la nuestra.

Figure 29: Logo JavaSpring

## 3.3 Desafíos y Soluciones

Al principio, lo que hicimos fue hacer la relación Title con Person, sin mirar los atributos ni nada, solo con las IDs y ya. Vimos que los campos Genre y Country no los leía MySQL, entonces nos obligó hacer más tablas de ambos campos.

Lo más complicado que hemos hecho en este trabajo ha sido tratar bien los atributos de Titles y darnos cuenta de que necesitabamos hacer muchas tablar para poder hacer bien nuestra BD con sus entidades y relaciones.
Gracias a JavaSpring, nos ha facilitado muchas cosas a la hora de gestionar la BD desde Java por lo que no nos ha sido muy difícil completarlo.

## 3.4 Resultados de la importación

Ahora vamos a mostrar como ha quedado nuestra base de datos al importar todos los .csv.
Empezamos con las tablas de las entidades:

| title_id | title | title_type | title_description | title_release_year | title_age_certification | title_runtime | title_seasons | title_imdb_id | title_imdb_score | title_imdb_votes | title_tmdb_popularity | title_tmdb_score |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| tm999916 | The Feast | MOVIE | A wealthy family hosts a sumptuous dinner, onl... | 2021 | | 93 | 0 | tt10738906 | 5.4 | 1791 | 3.131 | 6.2 |
| tm999927 | A Cop Movie | MOVIE | This documentary walks the line between fact a... | 2021 | R | 107 | 0 | tt14035242 | 7 | 1739 | 10.354 | 7.3 |
| tm999934 | kid 90 | MOVIE | As a teenager in the '90s, Soleil Moon Frye carri... | 2021 | PG-13 | 71 | 0 | tt14035292 | 6.8 | 2875 | 6.559 | 7.5 |
| tm99996 | The Owl and the Pussycat | MOVIE | Meek, owlish Felix and strident, catty Doris live i... | 1970 | PG | 95 | 0 | tt0066195 | 6.4 | 3929 | 3.448 | 6.3 |
| tm999993 | Gaia | MOVIE | A park ranger takes shelter with two survivalist... | 2021 | R | 97 | 0 | tt11881160 | 5.5 | 4685 | 15.021 | 6 |
| tm999994 | I'm Your Man | MOVIE | Alma is a scientist at Berlin's famous Pergamon ... | 2021 | R | 108 | 0 | tt13087796 | 7.1 | 6668 | 11.376 | 7 |
| ts1 | Star Wars: The Clone Wars | SHOW | Yoda, Obi-Wan Kenobi, Anakin Skywalker, Mace... | 2008 | TV-PG | 23 | 7 | tt0458290 | 8.4 | 85602 | 83.858 | 8.5 |
| ts10 | Homeland | SHOW | CIA officer Carrie Mathison is tops in her field d... | 2011 | TV-MA | 52 | 8 | tt1796960 | 8.3 | 335014 | 44.393 | 7.5 |
| ts1000 | Alice in Wonderland | SHOW | One of the most well-known stories begins one ... | 1986 | | 29 | 1 | tt0421576 | 7.2 | 73 | 0.6 | 7 |
| ts100116 | Blown Away | SHOW | A reality show featuring glass blowers from aro... | 2019 | TV-PG | 26 | 3 | tt9908860 | 7.1 | 3034 | 23.841 | 7.2 |
| ts10018 | Blackadder | SHOW | Through four series and a few specials, Edmund... | 1983 | TV-PG | 30 | 4 | tt0084988 | 8 | 37902 | 21.242 | 7.9 |
| ts100341 | Cheerleader Generation | SHOW | Follows the lives of two squads and their fierce,... | 2019 | TV-PG | 41 | 1 | tt9854758 | 7.2 | 52 | 1.215 | 9.7 |
| ts100344 | Marrying Millions | SHOW | Marrying Millions follows six couples who, althou... | 2019 | TV-14 | 41 | 2 | tt9854774 | 5 | 187 | 3.728 | 7.5 |

Figure 30: Tabla Title con los .csv importados

| person_id | title_id | person_name | person_character | person_role |
|---|---|---|---|---|
| 138 | ts343492 | Takahiro Sakurai | Yoshida (voice) | ACTOR |
| 139 | ts27027 | Keitaro Moton... | | DIRECTOR |
| 140 | ts217045 | Ayane Sakura | Natalia Garnet (voice) | ACTOR |
| 142 | ts86164 | Yuko Kaida | Sarah Sinclair (voice) | ACTOR |
| 143 | ts21740 | Shinnosuke Ta... | Soma | ACTOR |
| 144 | ts33329 | Misuzu Togashi | Origami Tobiichi (voice) | ACTOR |
| 145 | ts241854 | Rie Kugimiya | Happy (voice) | ACTOR |
| 146 | tm156894 | Christopher De... | Peppermint Patty (voice) | ACTOR |
| 149 | tm97573 | Bill Melendez | | DIRECTOR |
| 150 | tm921369 | Boone Smith | Himself | ACTOR |
| 152 | tm96153 | Sue Houghton | | DIRECTOR |
| 155 | tm827783 | Jadiel Dowlin | Prince Bryson | ACTOR |
| 156 | tm387 | Nico Cortez | Spanish Officer | ACTOR |
| 157 | tm315344 | Jacob Blair | Jeremy Bloom | ACTOR |

Figure 31: Tabla Person con los .csv importados

| role_id | role_name |
|---|---|
| 1 | ACTOR |
| 2 | DIRECTOR |

Figure 32: Tabla Role con los .csv importados

| genre_id | genre_name |
|---|---|
| 1 | comedy |
| 2 | family |
| 3 | animation |
| 4 | action |
| 5 | fantasy |
| 6 | horror |
| 7 | drama |
| 8 | war |
| 9 | western |
| 10 | european |
| 11 | romance |
| 12 | thriller |
| 13 | crime |
| 14 | history |
| 15 | sport |
| 16 | scifi |
| 17 | documentation |
| 18 | music |
| 19 | reality |

Figure 33: Tabla Genre con los .csv importados

| country_id | country_name |
|---|---|
| 1 | US |
| 2 | GB |
| 3 | MX |
| 4 | CA |
| 5 | DE |
| 6 | SU |
| 7 | IN |
| 8 | XX |
| 9 | IT |
| 10 | JP |
| 11 | FR |
| 12 | HK |
| 13 | ES |
| 14 | IL |
| 15 | AU |
| 16 | CH |
| 17 | IE |
| 18 | GR |
| 19 | CN |

Figure 34: Tabla Country con los .csv importados

| streaming_id | streaming_name |
|---|---|
| 1 | Amazon_Prime |
| 2 | Disney_Plus |
| 3 | HBOMax |
| 4 | HuluTV |
| 5 | Netflix |
| 6 | ParamountTV |
| 7 | Rakuten_Viki |

Figure 35: Tabla Streaming con los .csv importados

| title_id | title | title_type | title_description | title_release_year | title_age_certification | title_runtime | title_seasons | title_imdb_id | title_imdb_score | title_imdb_votes | title_tmdb_popularity | title_tmdb_score |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| tm37175 | Jeff Dunham: Minding the... | MOVIE | Stand-up comedian and ventriloquist Jeff Dunh... | 2012 | PG-13 | 80 | 0 | tt2461736 | 7 | 1416 | 2.314 | 6.2 |
| tm37945 | Rock On!! | MOVIE | Aditya, Joe, Kedar and Rob form a rock band, b... | 2008 | | 144 | 0 | tt1230165 | 7.7 | 22092 | 4.956 | 6.9 |
| tm38454 | Jeff Dunham's Very Speci... | MOVIE | The multi-platinum selling comedian performs his... | 2008 | PG-13 | 85 | 0 | tt1295909 | 7.4 | 3042 | 2.581 | 6.9 |
| tm39828 | Semi-Pro | MOVIE | Jackie Moon is the owner, promoter, coach, an... | 2008 | R | 94 | 0 | tt0839980 | 5.8 | 82159 | 12.789 | 5.6 |
| tm40860 | Big Time Movie | MOVIE | In Big Time Movie, the guys must tap their inner... | 2012 | G | 65 | 0 | tt2163203 | 6.1 | 1936 | 11.432 | 7.6 |
| tm409508 | Along with the Gods: The... | MOVIE | In the afterlife, one guardian helps a man throu... | 2018 | | 142 | 0 | tt8116428 | 7.1 | 7063 | 54.446 | 8.3 |
| tm412937 | Trading Paint | MOVIE | A stock car racing legend is drawn back to the d... | 2019 | R | 87 | 0 | tt7052494 | 4.4 | 1996 | 8.786 | 5.6 |
| tm414073 | Backtrace | MOVIE | The lone surviving thief of a violent armored car... | 2018 | R | 92 | 0 | tt3588588 | 3.8 | 4453 | 19.984 | 5.4 |
| tm41792 | Django Unchained | MOVIE | With the help of a German bounty hunter, a fre... | 2012 | R | 165 | 0 | tt1853728 | 8.4 | 1472670 | 56.588 | 8.1 |
| tm41953 | Tim Minchin: So F**king R... | MOVIE | Comedic pianist Tim Minchin performs a host of ... | 2008 | | 127 | 0 | | 0 | 0 | 2.714 | 6.9 |
| tm423436 | Ben Is Back | MOVIE | 19-year-old Ben Burns unexpectedly returns ho... | 2018 | R | 103 | 0 | tt7545524 | 6.7 | 21735 | 12.478 | 6.6 |
| tm426638 | Vita & Virginia | MOVIE | Socialite Vita Sackville-West and literary icon Vir... | 2019 | | 110 | 0 | tt5859882 | 5.7 | 3823 | 8.037 | 6.9 |

Figure 36: Tabla TitleDuplicated con los .csv importados

Donde se puede ver que en las tablas **Title**, **Person** y **TitleDuplicated**, la ID son importados de los .csv y las IDs de las tablas **Role**, **Genre**, **Country** y **Streaming** son generados automaticamente al insertar un nuevo elemento en la tabla.
También se pueden ver que hay campos que están vacíos, ya que no hay datos a la hora de importar los .csv.

Ahora vamos a mostrar las tablas de las relaciones:

| person_id | title_id | role_id | title_person_id |
|---|---|---|---|
| 146 | tm156894 | 1 | 216049 |
| 149 | tm1236 | 1 | 216034 |
| 149 | tm1236 | 2 | 216039 |
| 149 | tm156894 | 1 | 216041 |
| 149 | tm156894 | 2 | 216051 |
| 149 | tm97573 | 1 | 19440 |
| 149 | tm97573 | 2 | 19452 |
| 150 | tm921369 | 1 | 144468 |
| 152 | tm96153 | 2 | 138932 |
| 155 | tm827783 | 1 | 147812 |
| 155 | ts35648 | 1 | 75060 |

Figure 37: Tabla TitlePersonRole con los .csv importados

Aquí podemos ver varias cosas, empezando por la **Person** con la ID **149**, que ha participado en varios **Titles** como **Actor** (roleId=1) y como **Director** (roleId=2). El campo **titlePersonId** sirve para identificar la relación y que no se repita dicha relación en la misma tabla.

| title_id | genre_id | title_genre_id |
|---|---|---|
| tm1 | 4 | 22202 |
| tm1 | 5 | 22203 |
| tm1 | 16 | 22201 |
| tm10 | 4 | 28087 |
| tm10 | 16 | 28086 |

Figure 38: Tabla TitleGenre con los .csv importados

Aquí podemos observar que el **Title** con la ID **tm1** tiene 3 **Genres** que, viendo la tabla de **Genre**, podemos ver que son: **Action**, **Fantasy** y **Scifi**. Después tenemos el campo de **titleGenreId** para identificar sus relaciones.

| title_id | country_id | title_country_id |
|---|---|---|
| tm1000022 | 19 | 9988 |
| tm1000037 | 5 | 24015 |
| tm1000037 | 46 | 24014 |
| tm1000070 | 2 | 17957 |
| tm1000070 | 16 | 17959 |
| tm1000070 | 33 | 17958 |
| tm1000070 | 45 | 17956 |
| tm1000070 | 46 | 17960 |
| tm1000070 | 96 | 17955 |
| tm1000095 | 1 | 15694 |

Figure 39: Tabla TitleCountry con los .csv importados

Aquí podemos observar que el **Title** con la ID **tm1000070** tiene 6 **Countries** que, viendo la tabla de **Country**, podemos ver que son: **GB**, **CH**, **RO**, **LU**, **CZ** y **HR**. Después tenemos el campo de **titleCountryId** para identificar sus relaciones.

| title_id | streaming_id | streaming_title_id |
|---|---|---|
| tm1 | 2 | 9954 |
| tm10 | 3 | 12136 |
| tm100001 | 1 | 320 |
| tm100001 | 6 | 23162 |
| tm1000022 | 1 | 9405 |

Figure 40: Tabla TitleStreaming con los .csv importados

Aquí podemos observar que el **Title** con la ID **tm100001** tiene 2 **Streamings** que, viendo la tabla de **Streaming**, podemos ver que son: **Amazon Prime** y **ParamountTV**. Después tenemos el campo de **streamingTitleId** para identificar sus relaciones.

# 4 Fase 3: Construcción de una Red Cinematográfica

## 4.1 Diseño de la Base de Datos Orientada a Grafos

Para hacer el diseño de la Base de Datos orientada a Grafos, nos hemos decantado por utilizar **TigerGraph** que es un buen analizador de grafos el cual nos proporciona herramientas y funcionalidades para analizar las conexiones dentro de la base de datos de grafos.

Figure 41: Logo TigerGraph

Para la creación del Grafo, nos pedian que hubieran 2 nodos llamados **Person** y **ShowMovie**, los cuales estarán relacionados por 2 arestas llamadas **ACTED-IN** y **DIRECTED**.

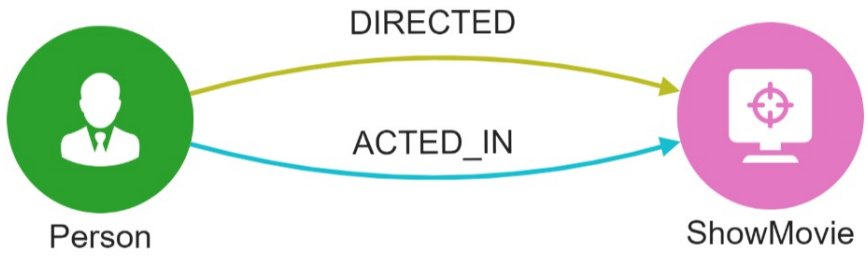


Figure 42: Estructura principal del Grafo

Para el nodo **Person**, guardamos el nombre juntamente con su ID.

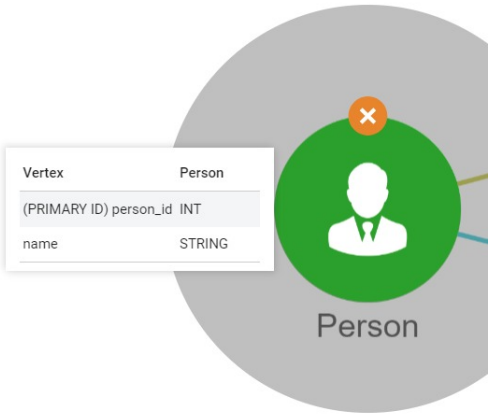


Figure 43: Nodo Person

Para el nodo **ShowMovie**, guardamos el nombre y el año de lanzamiento juntamente con su ID.

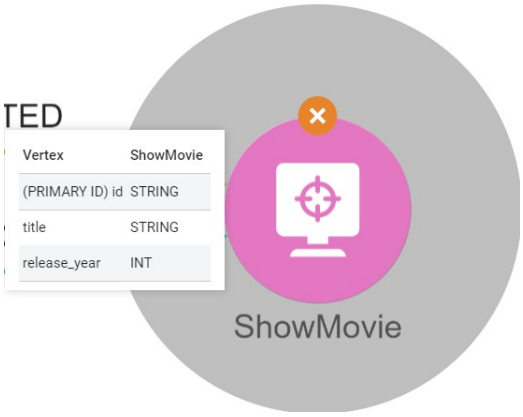


Figure 44: Nodo ShowMovie

Luego, en las aristas, decidimos que sean directas de **Person** a **ShowMovie** ya que en los atributos de **Person**, tenemos las IDs de los **Titles** pero en los atributos de **Title** no tienemos las IDs de los **Persons**.
En el caso de la arista **DIRECTED** solo almacenamos el rol de la **Person** que siempre será **Director** y en la otra arista **ACTED-IN**, almacenamos el rol que siempre es **Actor** y el nombre del personaje que actuó en la **ShowMovie** que participó.

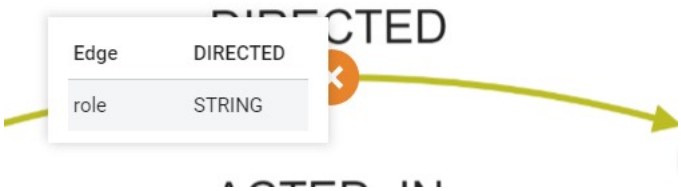


Figure 45: Arista Directed

Figure 46: Arista ActedIn

## 4.2 Importación de Datos

Para el proceso de importación de los datos se ha optado por seguir un proceso bastante similar a las anteriores fases, añadiendo por **GSQL** la especificación tanto de los nodos **Vertex** como de las relaciones **Edge**, de esta forma aprovechamos el proceso de la lógica de importación de datos anterior para esta fase.

```
@Bean
public CommandLineRunner lineReader() {
    final RestTemplate restTemplate;
    final TigerGraphService tigerGraphService = new TigerGraphService(restTemplate = new RestTemplate());
    return args -> {
        String filePath = "src/main/resources/movies.csv";
        try (FileReader fileReader = new FileReader(filePath)) {
            List<Title> movies = new CsvToBeanBuilder<Title>(fileReader)
                    .withType(Title.class) CsvToBeanBuilder<Title>
                    .build() CsvToBean<Title>
                    .parse();
            tigerGraphService.importTitlesToTigerGraph(movies);

            List<Person> persons = new CsvToBeanBuilder<Person>(fileReader)
                    .withType(Person.class) CsvToBeanBuilder<Person>
                    .build() CsvToBean<Person>
                    .parse();
            tigerGraphService.importPersonsToTigerGraph(persons);
        } catch (Exception e) {
```

Figure 47: Obtención del Token de Autorización

```
http://localhost:8080/PRAC1/webresources/rest/api/v1/rental/01234567E                    Save
POST   a932c8d8a13d497d844259d795890fdd.i.tgcloud.io/restpp/requesttoken                 Send
Params  Authorization  Headers (8)  Body  Pre-request Script  Tests  Settings            Cookies
none  form-data  x-www-form-urlencoded  raw  binary  JSON                                Beautify
1 {"secret": "vutlof2r2ivucs80lha9fnf3lg696bk3", "lifetime": "100000"}

ody  Cookies  Headers (6)  Test Results        Status: 200 OK  Time: 386 ms  Size: 632 B  Save Response
Pretty  Raw  Preview  Visualize  JSON
1 {
2     "code": "REST-0000",
3     "expiration": 1710802206,
4     "error": false,
5     "message": "Generate new token successfully.\nWarning: Tigergraph Support cannot restore access to secrets/tokens for security reasons.
          Please save your secret/token and keep it safe and accessible.",
6     "token": "fs8solrig6mothft900gbhflb0g11i39"
7 }
```

Figure 48: Importación del CSV

```
// Crear los tipos de vértices
CREATE VERTEX PERSON(PRIMARY_ID id STRING, name STRING, role STRING)
CREATE VERTEX TITLE(PRIMARY_ID id STRING, name STRING, releaseYear INT)

// Crear los tipos de aristas
CREATE DIRECTED EDGE ACTED_IN(FROM PERSON, TO TITLE)
CREATE DIRECTED EDGE DIRECTED(FROM PERSON, TO TITLE)

// Crear el grafo
CREATE GRAPH MovieGraph(PERSON, TITLE, ACTED_IN, DIRECTED)
```

Figure 49: Script GSQL

Es importante recalcar que para **TigerGraph** no disponemos de un gestor automático como el que tiene **JavaSpring** con **JDBC**, por ende haremos una conexión manual al puerto donde levantamos nuestro servidor **TigerGraph**.
Para ello, se levantará un cluster de **TigerGraph**, el cual será inicializado con el script **GSQL** de la Figure 49.
Con ello, se habrá inicializado el cluster al que se desea conectar.
A continuación, se debe obtener la request token a partir de una solicitud **POST** al endpoint que **TigerGraph** ofrece, en la Figure 47 se obtiene la request token a partir del **secret** que se especifica en **TigerGraph**.

```
1 usage
public void importTitlesToTigerGraph(List<Title> titles) {
    String endpoint = "/graph/yourGraph/vertices/Title";
    for (Title title : titles) {
        String json = gson.toJson(title);
        restTemplate.postForObject( url: TIGERGRAPH_URL + endpoint, json, String.class);
    }
}

1 usage
public void importPersonsToTigerGraph(List<Person> persons) {
    String endpoint = "/graph/yourGraph/vertices/Person";
    for (Person person : persons) {
        String json = gson.toJson(person);
        restTemplate.postForObject( url: TIGERGRAPH_URL + endpoint, json, String.class);
    }
}
```

Figure 50: Uso de RestTemplate para hacer queries a TigerGraph

Con eso obtenemos la cookie de autorización, y con esa cookie desde spring podemos configurar las solicitudes **POST** que vayamos a hacer con dicha cookie.
Con esa configuración hecha, se podrá usar la aplicación spring que permite persistir las entidades que conforman el grafo, en la Figure 47 se encuentra la lógica que permite extraer las entidades **Person** y **Title** de los archivos csv, en la Figure 50 se encuentra la lógica que se emplea para poder hacer la query al clúster del grafo.